

TANESCO FAULT MANAGEMENT SYSTEM

SYSTEM DOCUMENTATION AND DIAGRAMS

A web-based system for reporting, tracking, assigning, managing, resolving, and closing customer electrical fault complaints.

Technology Stack	Java, Spring Boot, Spring Security, JWT, MySQL, HTML, CSS, JavaScript
Main Roles	Customer, Officer, Technician, Administrator
Architecture	Layered REST-based web application
Core Workflow	SUBMITTED -> ASSIGNED -> IN_PROGRESS -> RESOLVED -> CLOSED

Prepared as technical system documentation for the implemented project.

Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Scope and User Roles
5. Functional Requirements
6. Non-Functional Requirements
7. System Architecture
8. System Diagrams
9. Database Design
10. Security Design
11. Implementation
12. Testing and Verification
13. Limitations and Future Improvements
14. Conclusion

1. Introduction

The TANESCO Fault Management System is a web-based application developed to support the management of customer electrical fault complaints. It provides a structured process through which customers can report faults, officers can review and assign faults to technicians, technicians can work on and resolve assigned faults, and customers can confirm that the reported problem has been resolved. Administrators manage staff accounts and obtain system-level information.

2. Problem Statement

Fault complaints require clear tracking from the time they are reported until they are resolved and closed. Without a centralized workflow, it can be difficult to know whether a complaint has been assigned, whether technical work has started, who is responsible for the fault, and whether the customer has confirmed restoration. The proposed system addresses this by maintaining a traceable fault lifecycle and role-based responsibilities.

3. Objectives

The main objective is to provide a centralized digital platform for managing electrical fault complaints. Specific objectives are to allow customers to report and track faults; enable officers to assign technicians and monitor progress; allow technicians to manage assigned work and record resolutions; enable customers to confirm resolution; maintain fault history and evidence metadata; and provide administrators with staff-management capabilities.

4. Scope and User Roles

The implemented system contains four roles. Customers register themselves and report faults. Officers view faults and assign technicians. Technicians view assigned faults, start work, update status, upload supporting evidence, and resolve faults. Administrators create and view staff accounts and access administrative information. The application uses separate role-based web portals while sharing the same Spring Boot backend and MySQL database.

Role	Main Responsibilities
Customer	Register/login; report faults; view own faults; track status; view details; confirm resolution.
Officer	Login; view/search/filter faults; inspect details; assign technicians; monitor progress.
Technician	Login; view assigned faults; start work; update status; upload evidence; record cause and resolution; resolve faults.
Administrator	Login; view dashboard; view staff; create OFFICER, TECHNICIAN and ADMIN accounts.

5. Functional Requirements

The system must authenticate users, enforce role-based authorization, allow customer registration, create unique fault records, support priority/category/location/description information, allow staff to query faults, assign technicians, change fault status through the permitted workflow, store resolution information, record fault updates, accept validated evidence attachments, allow customers to confirm resolution, and support administrator creation of staff accounts.

6. Non-Functional Requirements

The system should provide secure password storage, stateless authenticated API access, responsive user interfaces, maintainable layered code, validation of incoming data, controlled file uploads, reliable database persistence, understandable error responses, and a workflow that is easy for each role to follow.

7. System Architecture

The application follows a layered web architecture. Browser-based portals communicate with REST endpoints in the Spring Boot backend. Controllers receive HTTP requests, services implement business rules, repositories perform persistence through Spring Data JPA/Hibernate, and MySQL stores application data. Spring Security, JWT, BCrypt, validation, and access-control rules protect backend operations.

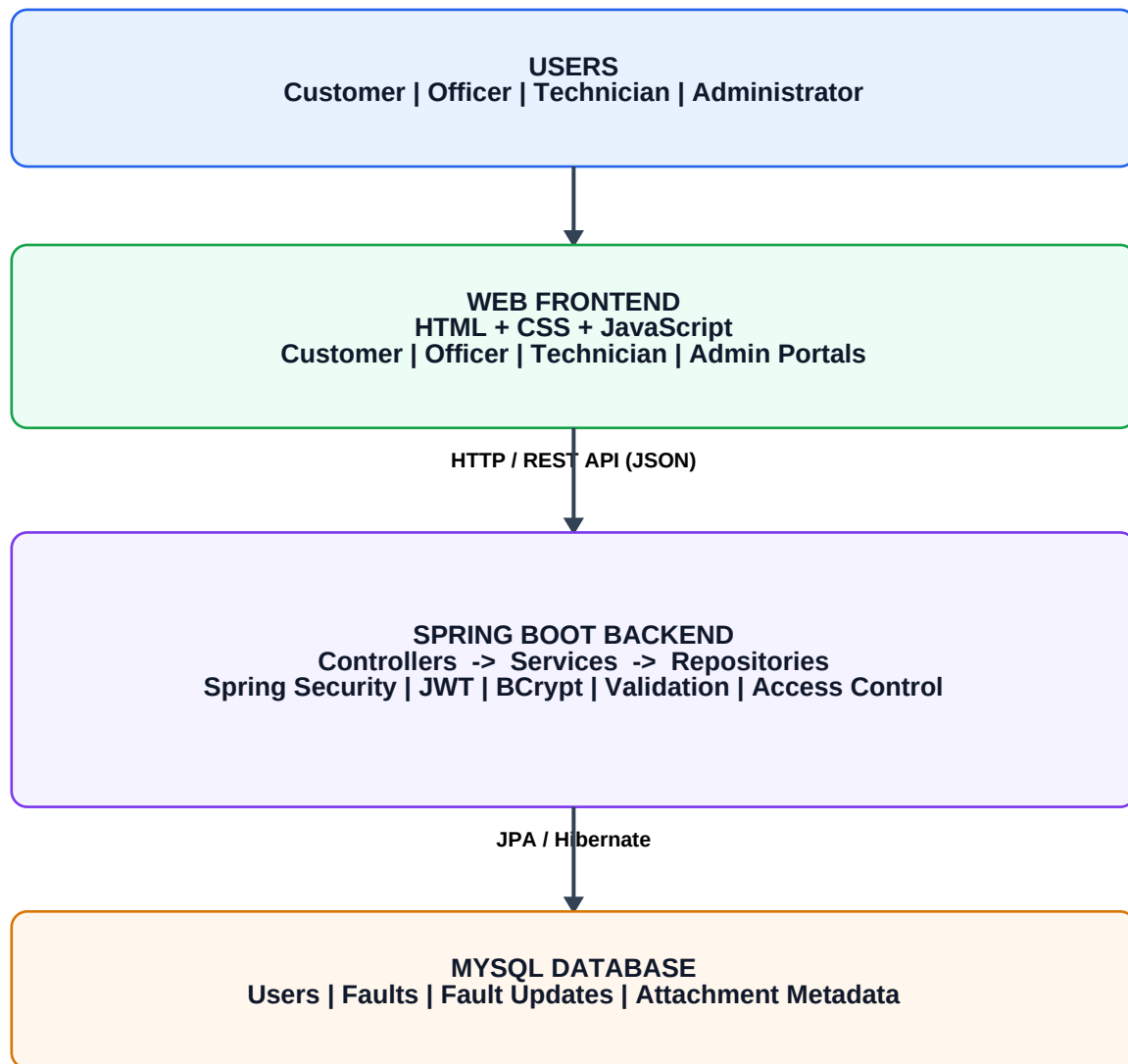


Figure 1: System Architecture Diagram

8. System Diagrams

8.1 Use Case Diagram

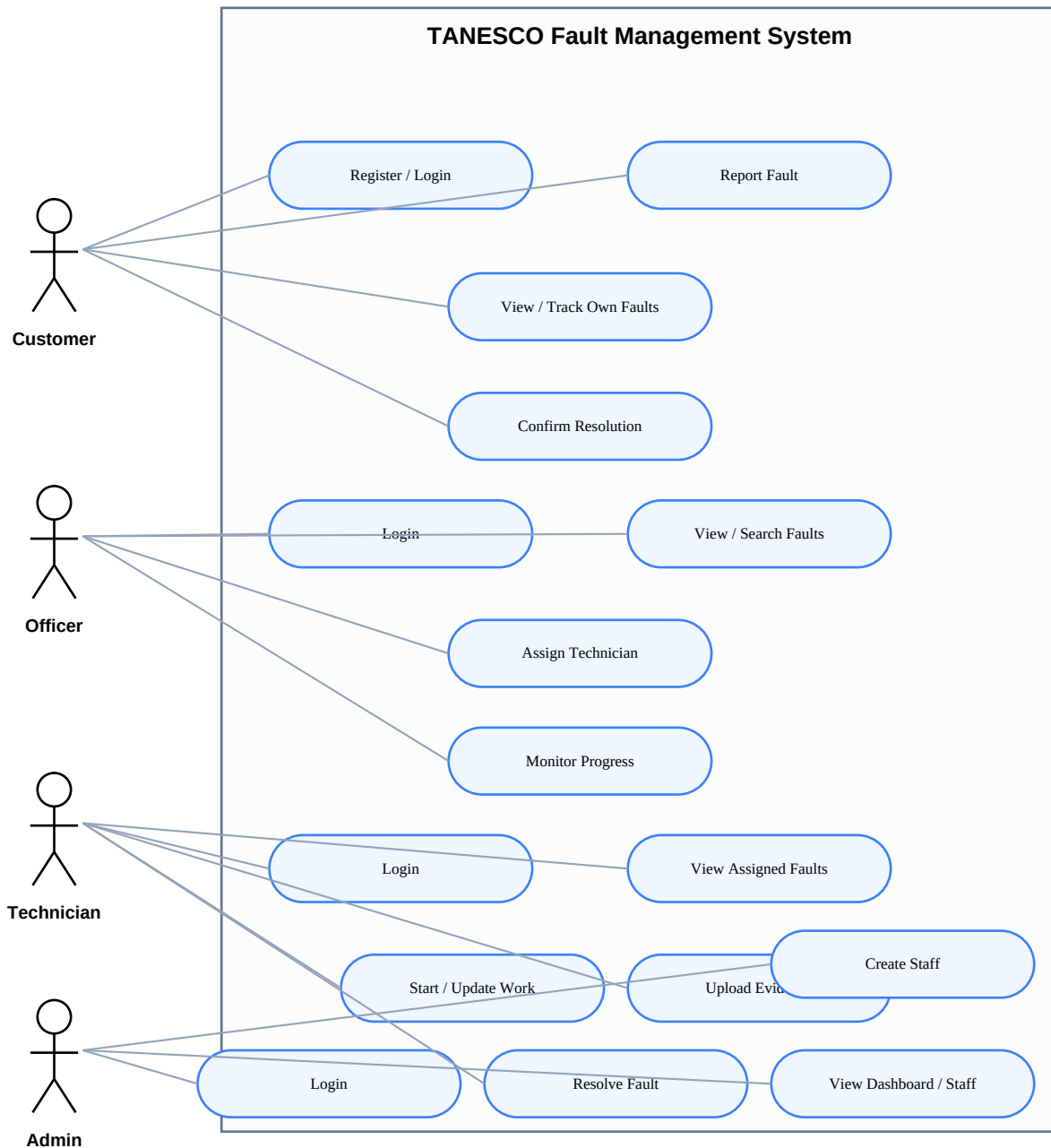


Figure 2: Use Case Diagram

8.2 Fault Lifecycle / State Diagram



Figure 3: Fault Lifecycle / State Diagram

8.3 Activity Diagram

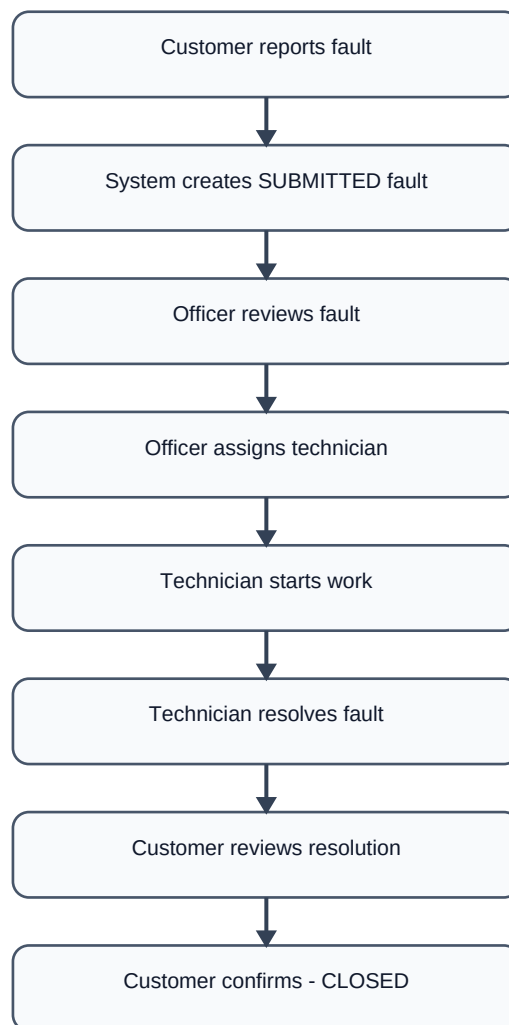


Figure 4: Main Fault Management Activity Diagram

8.4 Sequence Diagram

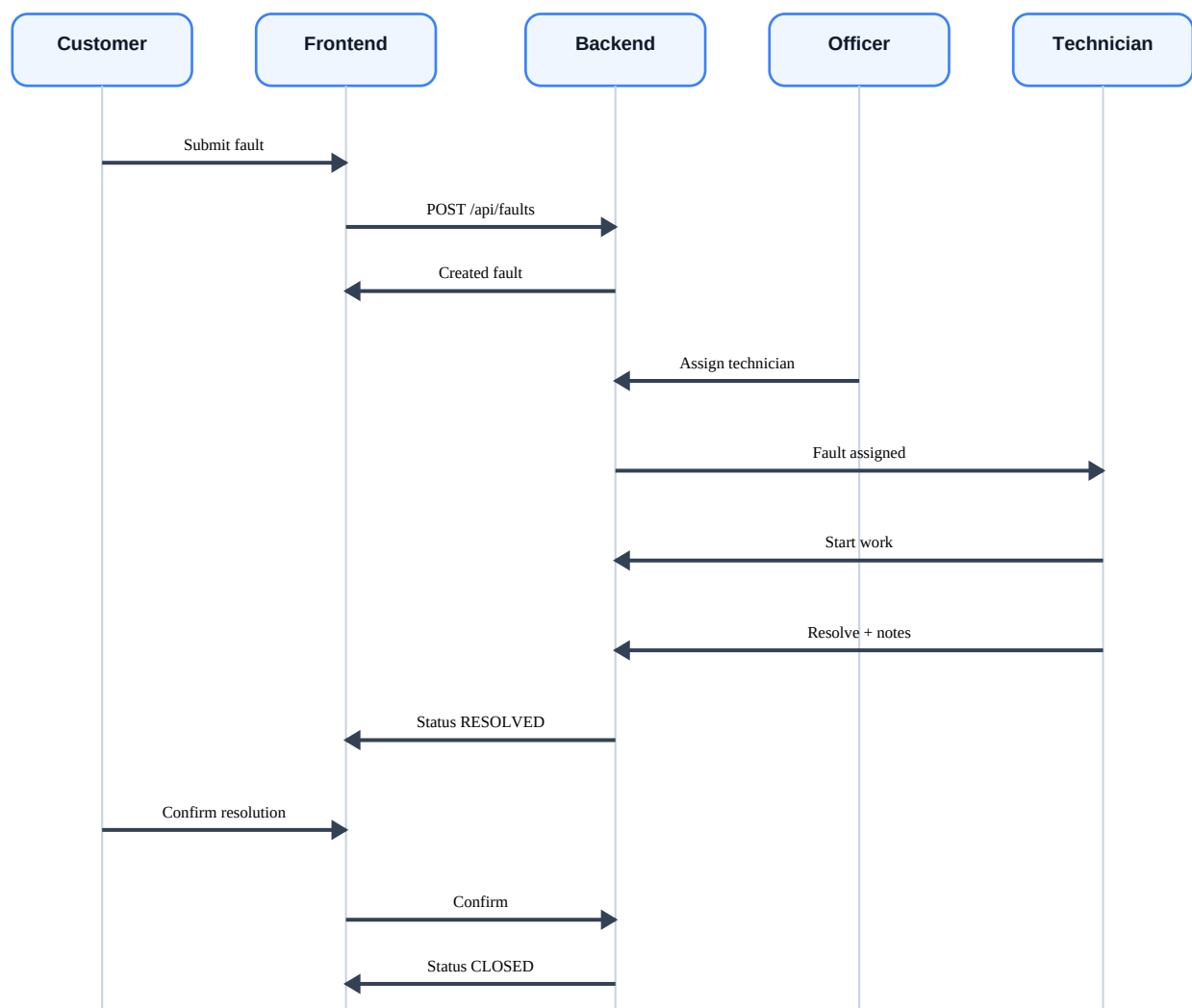


Figure 5: Simplified End-to-End Sequence Diagram

8.5 Class Diagram

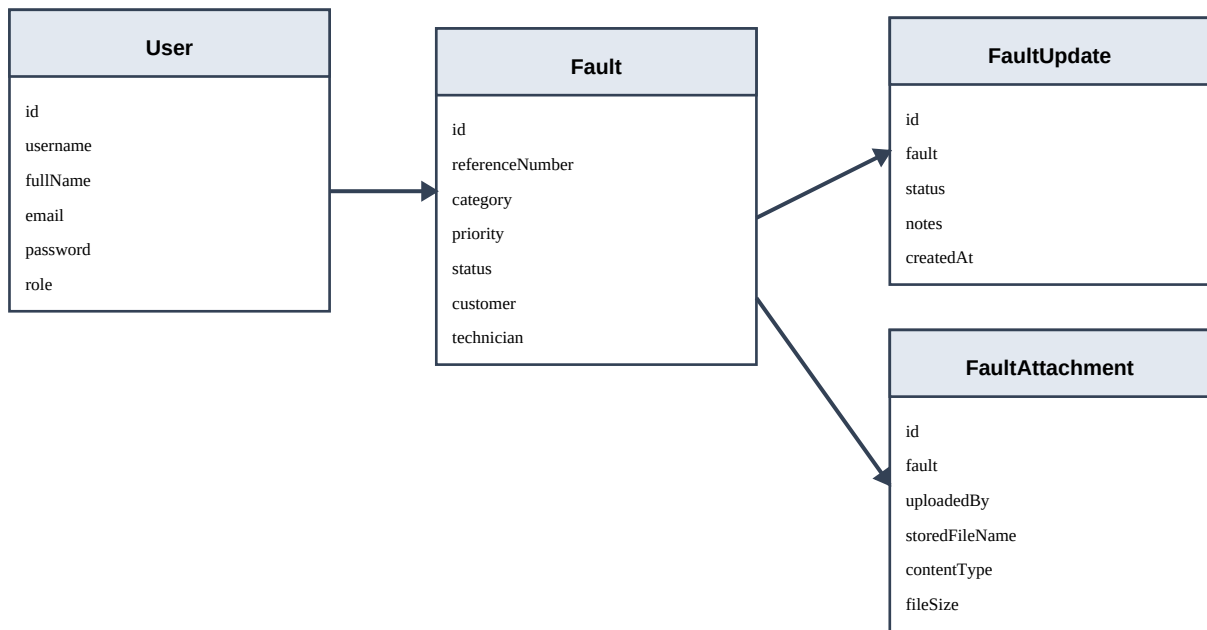


Figure 6: Simplified Domain Class Diagram

9. Database Design

The database design is centered on users and faults. A user has a role and may act as a customer, officer, technician, or administrator. Each fault belongs to a customer and may be assigned to a technician. Fault updates maintain status/history information, while attachment records maintain metadata for evidence files.

9.1 Entity-Relationship Diagram

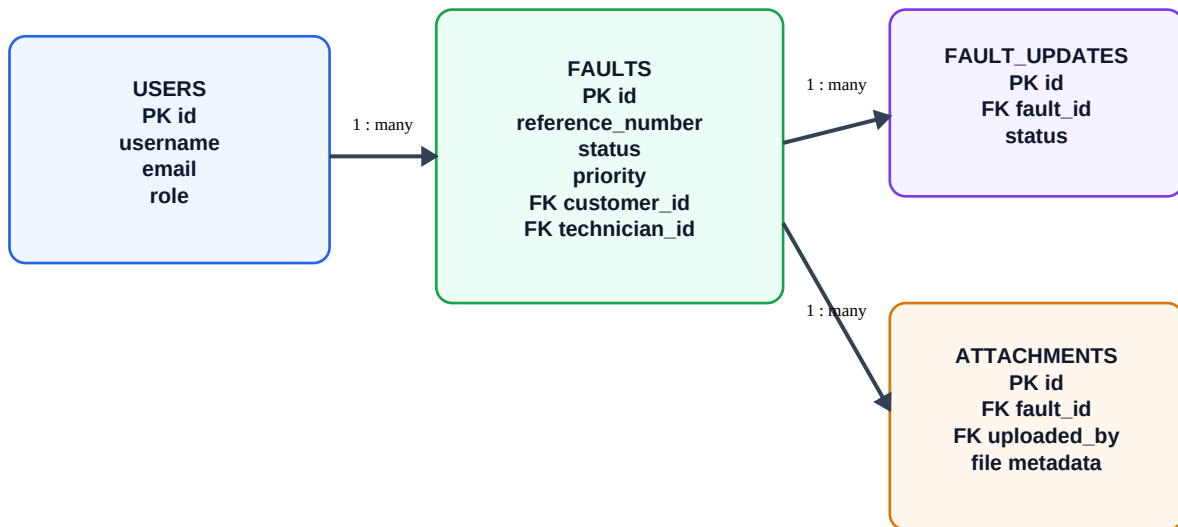


Figure 7: Simplified Entity-Relationship Diagram

10. Security Design

Authentication is JWT-based and stateless. Passwords are encoded using BCrypt. Spring Security protects API routes according to roles, while method-level security is enabled for additional authorization rules. The JWT service signs tokens, validates signatures, checks token subjects, and rejects expired tokens. Uploaded evidence is restricted by size and allowed content type, and access checks prevent customers or technicians from accessing fault evidence outside their permitted scope.

11. System Implementation

The backend is implemented in Java using Spring Boot. Controllers expose REST endpoints; service classes implement business logic; repositories use Spring Data JPA; Hibernate maps entities to MySQL. The frontend uses HTML, CSS, and JavaScript and contains separate portals for the four roles. Maven manages dependencies, compilation, testing phases, and packaging.

Layer / Component	Implementation
Frontend	HTML5, CSS3, JavaScript; responsive role-based pages.
API	REST-style JSON endpoints served by Spring Boot.
Business Logic	Service classes for faults, staff, dashboard, history, attachments and resolution.
Persistence	Spring Data JPA repositories with Hibernate and MySQL.
Security	Spring Security, JWT authentication, BCrypt password encoding, role authorization.

Layer / Component	Implementation
Build	Maven; executable Spring Boot JAR.

12. Testing and Verification

The implemented system was manually verified through a complete end-to-end scenario. A customer created fault TAN-2026-0917-5858 with status SUBMITTED. An officer assigned technician tech01, changing the status to ASSIGNED. The technician started work, producing IN_PROGRESS, then resolved the fault, producing RESOLVED. The original customer confirmed the resolution and the final status became CLOSED. This verified the main cross-role lifecycle.

The project was also compiled with Maven using 'mvn clean test' and packaged with 'mvn clean package'. The clean build succeeded and an executable Spring Boot JAR was produced. Maven reported that there are currently no automated test sources, so the present verification is primarily manual functional testing.

Test Area	Result
Customer fault reporting	Passed
Officer technician assignment	Passed
Technician start work	Passed
Technician resolution	Passed
Customer confirmation / closure	Passed
Role-based authentication and access	Passed during functional review
Maven clean build	BUILD SUCCESS
Executable JAR packaging	BUILD SUCCESS
Automated JUnit tests	Not currently implemented

13. Limitations and Future Improvements

Current improvement opportunities include automated unit and integration tests, customer viewing/downloading of technician evidence, email/SMS notifications, stronger escalation mechanisms for unattended faults, advanced reporting and analytics, geographic visualization, password recovery, audit logging, cloud/object storage for evidence, and production deployment with environment-based secrets and production CORS settings.

14. Conclusion

The TANESCO Fault Management System implements a complete role-based workflow for electrical fault complaint management. It connects customers, officers, technicians, and administrators through a centralized web application and maintains fault progression from submission to closure. The implemented architecture, security controls, database persistence, responsive frontend, and successful end-to-end workflow provide a solid foundation for further enhancement and deployment.